Architecture Proof Notes for Modern LLM Families

Fractal Research Notes

April 6, 2026

Abstract

These notes collect the smallest clean theorem surface for four architecture families:

1. standard decoder-only language models,
2. modern sub-variants including sparse mixture-of-experts, linear attention, and hybrid recurrent-attention stacks such as Kimi Linear,
3. the current `A + P2` contender family,
4. a speculative Graph-of-Experts / thought-channel family for bounded internal search.

The aim is not to prove empirical superiority. The aim is to isolate claims that are actually provable: causality, incremental equivalence, reduction to simpler cases, strict structural generalization where valid, and asymptotic or control-plane bounds.

1 Proof Discipline and Common Machine

Remark 1.1. *Every formal statement in this note should be interpreted as exactly one of:* definition*,* lemma*,* theorem*,* proposition*, or* conjecture*. If benchmark evidence is required, the statement is not a theorem.*

Let the input token sequence be

$$x_1, x_2, \ldots, x_T.$$

At step t, define:

- h_t: active latent state,
- M_t: memory object,
- c_t: control object,
- y_t: emitted token representation used by the output head.

We place all families inside the generic causal machine

$$(h_t, M_t, c_t) = U_\theta(h_{t-1}, M_{t-1}, c_{t-1}, x_t), \tag{1}$$

$$y_t = R_\theta(h_t, M_t, c_t, x_t), \tag{2}$$

$$p_\theta(x_{t+1} \mid x_{\leq t}) = \mathrm{Softmax}(W y_t). \tag{3}$$

What varies across architectures is:

- what M_t stores,
- what c_t controls,
- whether U_θ is attention-dominant, recurrent-dominant, hybrid, or search-native,
- how much exact token interaction is available inside U_θ.

1.1 Shared Symbols

- T: sequence length,
- d: model width,
- w: local exact-attention window,
- L: total layer count,
- L_A: exact-attention layer count,
- L_S: sequence-mixer layer count,
- E: number of experts in a sparse MoE layer,
- k: number of active experts per token,
- K: number of active thought channels,
- B: bounded internal search budget,
- s_t: recurrent latent state in a sequence-mixer block.

1.2 Basic Causality

Definition 1.2 (Causal update)**.** *An update rule is* causal *if step t depends only on $x_{\leq t}$ and state produced from steps $< t$, and never on any future token x_u with $u > t$.*

Lemma 1.3 (Causal composition)**.** *If f and g are causal update operators over the same prefix order, then their composition $g \circ f$ is causal.*

Proof. The output of f depends only on allowed prefix information. The output of g depends only on the output of f and the same allowed prefix information. Therefore no future token can enter through composition. □

Definition 1.4 (Incremental-decode equivalence)**.** *An incremental decode procedure is equivalent to full-prefix recomputation if, for fixed weights, masks, and prefix,*

$$y_t^{\text{inc}} = y_t^{\text{full}}$$

up to deterministic arithmetic tolerance.

Definition 1.5 (Reduction to a simpler class)**.** *Architecture family F reduces to family G if there exists an explicit parameter or control setting under which every member of the reduced subclass of F behaves as a member of G.*

Definition 1.6 (Strict structural generalization)**.** *Family F strictly structurally generalizes family G if:*

1. *F reduces to G under an explicit restriction, and*
2. *there exist members of F that cannot be represented inside G without changing the declared contract class.*

2 Standard Decoder LLMs

Assume masked causal self-attention, residual blocks, and position-wise feed-forward updates.

For one attention head:

$$q_t = W_Q r_t, \tag{4}$$
$$k_u = W_K r_u, \tag{5}$$
$$v_u = W_V r_u. \tag{6}$$

With causal masking:

$$\alpha_{t,u} = \begin{cases} 0, & u > t, \\ \dfrac{\exp(q_t \cdot k_u/\sqrt{d_k})}{\sum_{j \le t} \exp(q_t \cdot k_j/\sqrt{d_k})}, & u \le t. \end{cases}$$

Then

$$\mathrm{Attn}_t = \sum_{u \le t} \alpha_{t,u} v_u.$$

Definition 2.1 (Decoder language model)**.** *A standard decoder language model is any stacked residual architecture in which every layer is causal and the next-token distribution is read from the top layer token representation:*

$$p_\theta(x_{t+1} \mid x_{\le t}) = \mathrm{Softmax}(W r_t^{(L)}).$$

Theorem 2.2 (Causal decoder correctness)**.** *A masked decoder transformer defines a causal conditional distribution over the next token.*

Proof. At every layer, the causal mask forbids access to positions $u > t$. Feed-forward blocks are pointwise in position and therefore cannot introduce future-token dependence. Residual addition preserves causality whenever both branches are causal. By repeated application of the causal-composition lemma, the full stack remains causal, and the output head reads only the final causal state at position t. □

Theorem 2.3 (KV-cache decode equivalence)**.** *Under fixed weights, fixed masks, and deterministic inference, incremental decode with cached keys and values is equivalent to full-prefix recomputation for the same prefix.*

Proof. At step t, full-prefix recomputation attends to the set $\{(k_u, v_u) : u \le t\}$. Under fixed weights, the keys and values for prior positions $u < t$ do not change when a new token arrives. Therefore the cached set of prior key-value pairs is exactly the same set that full-prefix recomputation would rebuild. The only new objects required are the current token's key and value. Since the score function and mask are unchanged, the attention output is the same. □

Proposition 2.4 (Approximate copy capacity)**.** *Exact self-attention can implement arbitrarily sharp copy-like retrieval of a visible prior token.*

Proof. Choose parameters so that the query at position t scores one visible prior position $u^\star \le t$ strictly above all others. Then the attention weights concentrate on $u^\star$, and the resulting output becomes arbitrarily close to $v_{u^\star}$. If the output projection reads the copied content from $v_{u^\star}$, the model implements pointer-style copying over the visible context. □

Proposition 2.5 (Token comparison capacity)**.** *Exact self-attention can implement local or long-range token comparison over the visible context.*

Proof. Keys and queries can be parameterized so that the current token representation selects one or more comparison-relevant prior tokens. The attention read gathers their information into the residual stream, and a subsequent feed-forward block can map that gathered evidence to a comparison feature. □

Proposition 2.6 (Complexity bound)**.** *Let sequence length be T, model width be d, and layer count be L. Then train-time dense attention is quadratic in visible sequence length per attention layer, while decode-time incremental attention is linear in visible cache length per attention layer.*

Proof. At train time, each visible query position scores against all visible prior keys in each attention layer. That induces the familiar quadratic prefix interaction. At incremental decode time, each new query scores against the already-visible cache once per layer, which is linear in cache length for that step. □

3 Modern Sub-Variants

3.1 Sparse Mixture of Experts

Write one sparse-MoE feed-forward block as

$$\mathrm{MoE}(x_t) = \sum_{i \in \mathrm{Active}_t} \pi_{t,i} E_i(x_t),$$

where $|\mathrm{Active}_t| \leq k$.

Proposition 3.1 (Dense reduction for sparse MoE)**.** *A sparse-MoE layer reduces to a dense feed-forward layer under a degenerate expert-sharing and routing choice.*

Proof. Set every expert function equal to the same dense function F. Force the router to activate exactly one expert with weight 1. Then

$$\mathrm{MoE}(x_t) = F(x_t).$$

Hence dense FFN is a special case of MoE. □

Proposition 3.2 (Expert budget invariant)**.** *If routing activates at most k experts per token, then per-token expert compute is bounded by k expert evaluations.*

Proof. The router is the only component allowed to activate experts. By contract it emits at most k active experts for a token, so no more than k expert evaluations can occur for that token. □

Proposition 3.3 (Control-plane separation)**.** *Sparse MoE changes conditional compute allocation without changing the causal prefix contract of the surrounding decoder.*

Proof. Routing depends on the current causal token state, and expert evaluation consumes only that causal state. Therefore expert selection does not introduce future-token dependence by itself. □

Conjecture 3.4 (Parameter-efficiency benefit)**.** *Sparse expert specialization may improve parameter-efficiency. This is an empirical claim, not a theorem.*

3.2 Linear-Attention Families

For a factorized linear-attention family with feature map ϕ, define

$$S_t = \sum_{u \le t} \phi(k_u) v_u^\top, \tag{7}$$

$$z_t = \sum_{u \le t} \phi(k_u), \tag{8}$$

$$\text{output}_t = \frac{\phi(q_t)^\top S_t}{\phi(q_t)^\top z_t}. \tag{9}$$

Theorem 3.5 (Associative recurrent form)**.** *If the attention kernel factors through an associative feature map ϕ, then prefix processing admits an equivalent recurrent update with linear-time sequence evolution.*

Proof. The objects S_t and z_t can be updated from S_{t-1} and z_{t-1} using only the current token contribution. No revisit of all prior tokens is required once those prefix summaries are maintained. Therefore the visible-prefix read admits a recurrent implementation over bounded summary state. □

Remark 3.6 (Softmax exactness boundary)**.** *The factorized recurrent form above should not be read as a general exact-equivalence guarantee to arbitrary softmax attention. Establishing a sharp non-equivalence theorem would require a separate representation argument beyond the scope of these notes.*

Proposition 3.7 (Decode complexity bound)**.** *If the model maintains recurrent prefix summaries of bounded size, incremental decode cost is bounded by the recurrent-state update rather than a full visible-cache scan.*

Proof. The recurrent summaries replace explicit re-scanning of all visible prior tokens. Thus the decode cost is governed by summary update size rather than visible context length itself. □

3.3 Hybrid Recurrent-Attention Families

This class includes:

- Jamba-style interleaved attention and SSM hybrids,
- Nemotron-H-style mostly-SSM stacks with occasional attention,
- Kimi-style recurrent-memory plus periodic exact-attention stacks.

Let the stack contain L_A exact-attention layers and L_S recurrent or SSM sequence-mixer layers.

Definition 3.8 (Hybrid layer cost model)**.** *Let*

$$c_A(T, d), \quad m_A(T, d)$$

denote the per-layer compute and memory obligations of one exact-attention layer on visible context length T, and let

$$c_S(d), \quad m_S(d)$$

denote the per-layer compute and memory obligations of one recurrent or SSM layer with bounded summary state.

Theorem 3.9 (Hybrid causal composition)**.** *An interleaved stack of causal exact-attention and causal recurrent or SSM blocks remains causal.*

Proof. Each attention block is causal by masked attention. Each recurrent or SSM block is causal by left-to-right update. The full stack is causal by repeated composition. □

Proposition 3.10 (Exact-attention budget decomposition)**.** *Under the hybrid layer cost model, the exact-attention-specific portion of the stack cost is*

$$L_A \, c_A(T, d)$$

in compute and

$$L_A \, m_A(T, d)$$

in memory.

Proof. Only the exact-attention layers incur exact visible-token interaction costs. The recurrent or SSM layers replace those exact-attention obligations with their own bounded summary-state updates. Summing over L_A exact-attention layers yields the stated exact-attention compute and memory terms. □

Proposition 3.11 (Reduction edge cases)**.** *A hybrid recurrent-attention stack reduces to attention-only when $L_S = 0$, and to recurrent-only when $L_A = 0$.*

Proof. Setting one layer family count to zero removes that family entirely. The remaining stack is exactly the edge-case class. □

Proposition 3.12 (Kimi-style periodic exact-attention exposure)**.** *In a fixed* 3:1 *KDA-to-MLA schedule, an exact-attention layer occurs at regular depth intervals while recurrent memory carries the cheaper background state between those refresh points.*

Proof. Every fourth layer is an exact-attention layer in the stated schedule. Therefore any representation that traverses depth encounters an exact interaction surface at regular intervals, while the intervening KDA layers carry the recurrent-memory path. □

Proposition 3.13 (Hybrid memory decomposition)**.** *Under the hybrid layer cost model, total layer-local memory obligation is*

$$L_A \, m_A(T, d) + L_S \, m_S(d).$$

In particular, the exact visible-token memory term scales with L_A rather than the full layer count L.

Proof. The recurrent-memory layers replace full exact-attention visible-token state with bounded recurrent summaries. Therefore exact visible-token memory appears only in the L_A exact-attention layers, while the remaining L_S layers contribute only their bounded summary-state terms. □

4 `A + P2`

The `A + P2` family is the current Path 1 contender:

$$A\text{-}P2\text{-}A\text{-}P2\text{-}A\text{-}P2\text{-}A\text{-}P2.$$

4.1 Expected Contract Class

We assume the currently expected direction:

- predictive-core sequence primitive only,
- no memory/index sidecar behavior,
- transformed state dynamics before blending,
- emitted output distinct from latent state,
- no routed retrieval or Path 2 control-plane behavior.

Write the `P2` step abstractly as

$$u_t = T_\theta(s_{t-1}, x_t), \tag{10}$$
$$c_t = C_\theta(x_t), \tag{11}$$
$$s_t = U_\theta(u_t, c_t, x_t), \tag{12}$$
$$o_t = R_\theta(s_t, x_t). \tag{13}$$

Remark 4.1. *This section is intentionally conditional on the current expected `P2` contract class. It does not claim that every final implementation detail is already frozen.*

Definition 4.2 (Abstract `P1`-style contract)**.** *For these notes, a `P1`-style family is any single-state recurrent predictor of the form*

$$s_t = G_\theta(s_{t-1}, x_t), \tag{14}$$
$$y_t = s_t, \tag{15}$$

with no separate emitted object beyond the latent state itself.

Definition 4.3 (Step/scan consistency for `P2`)**.** *A `P2` implementation is step/scan consistent if repeated `step(...)` over a sequence and one `scan(...)` over the same sequence produce the same emitted outputs and final latent state, up to deterministic arithmetic tolerance.*

Theorem 4.4 (Causal `P2` update correctness)**.** *If T_θ, C_θ, U_θ, and R_θ depend only on x_t and s_{t-1}, then the `P2` block is causal.*

Proof. The transformed carry path sees only the prior latent state and current input. The candidate path sees only the current input. The update combines only those causal objects, and the readout sees only the resulting state and current input. No future token can enter the block. □

Theorem 4.5 (`A + P2` hybrid causality)**.** *If every A block is causal by masked local attention and every `P2` block is causal, then the interleaved `A + P2` stack is causal.*

Proof. Apply the causal-composition lemma layer by layer through the interleaved stack. □

Proposition 4.6 (`P1`-style reduction)**.** *Under explicit restrictions, the `P2` family reduces to a simpler `P1`-style single-state direct-readout family.*

Proof. Restrict R_θ to direct state readout so that $o_t = s_t$, absorb C_θ into the update map, and choose T_θ and U_θ so that their composition becomes one single-state recurrent map $G_\theta(s_{t-1}, x_t)$. Under those restrictions the `P2` step becomes

$$s_t = G_\theta(s_{t-1}, x_t), \qquad y_t = s_t,$$

which is exactly the declared `P1`-style contract. □

Proposition 4.7 (Structural enlargement over the abstract `P1` contract)**.** *If the final `P2` contract permits a separate emitted object $o_t = R_\theta(s_t, x_t)$ that is not required to equal s_t, then the expected `P2` class strictly structurally generalizes the abstract `P1`-style family defined above.*

Proof. The reduction proposition shows that `P2` contains a restricted `P1`-style subclass. But by definition the abstract `P1` contract requires $y_t = s_t$ for every step. Any `P2` instance with an emitted object $o_t \neq s_t$ on some reachable state trajectory cannot be represented inside that direct-readout contract without changing the contract class. Hence the expected `P2` class strictly enlarges the structure. □

Proposition 4.8 (Matched slot complexity)**.** *Assume both `M` and `P2` are bounded-state sequence primitives whose per-layer update costs are independent of full visible context length beyond the local-attention work already paid by A. Then interleaved `A + M` and `A + P2` stacks share the same slot-level asymptotic form: exact local-attention cost from A plus bounded sequence-primitive cost from the companion block.*

Proof. The A layers retain the exact local-attention obligation. The `P2` layers update bounded recurrent or sequence-mixer state instead of performing full context scans, and the same assumption holds for the reference `M` lane. Therefore both hybrids have the same slot-level asymptotic decomposition: attention work in A and bounded non-attention update work in the companion sequence primitive. □

Conjecture 4.9 (Efficiency-quality advantage)**.** *Whether `A + P2` matches or beats `A` or `A + M` in quality, memory, or throughput is empirical.*

5 Graph-of-Experts / Thought-Channel Model

The speculative Graph-of-Experts family contains:

- a shared exact-attention trunk,
- K active thought channels,
- per-channel recurrent or SSM workspaces,
- explicit compare, score, prune, merge, and halt control.

The goal is not classic sparse expert routing. The goal is bounded internal multi-hypothesis search.

At step t, define:

- h_t: shared trunk state,
- $F_t = \{f_t^{(1)}, \ldots, f_t^{(K_t)}\}$: active channel frontier,

- c_t: controller state.

One abstract transition form is:

$$h_t = \text{SharedAttention}_\theta(h_{t-1}, x_t), \tag{16}$$
$$F_t^{\text{seed}} = \text{Seed}_\theta(h_t, F_{t-1}, c_{t-1}), \tag{17}$$
$$F_t^{\text{expand}} = \text{Expand}_\theta(F_t^{\text{seed}}, h_t), \tag{18}$$
$$F_t^{\text{retrieve}} = \text{Retrieve}_\theta(F_t^{\text{expand}}, h_t), \tag{19}$$
$$F_t^{\text{cmp}} = \text{CompareMerge}_\theta(F_t^{\text{retrieve}}, h_t), \tag{20}$$
$$(F_t, c_t) = \text{Control}_\theta(F_t^{\text{cmp}}, c_{t-1}), \tag{21}$$
$$y_t = \text{Readout}_\theta(h_t, F_t, c_t). \tag{22}$$

Assume explicit bounds:

- $K_t \leq K_{\max}$,
- at most B internal refinement rounds per output step,
- controller operations preserve the channel cap.

Definition 5.1 (Channel-separated state)**.** *A thought-channel system has channel-separated state if each live hypothesis is assigned a distinct explicit state slot and any cross-channel interaction occurs only through declared compare or merge operators.*

Proposition 5.2 (Single-channel reduction)**.** *When $K_{\max} = 1$, branching is disabled, and merge and prune are trivial, the Graph-of-Experts family reduces to a single-stream hybrid predictor.*

Proof. With one channel there is no branching, compare and merge become vacuous, and the controller reduces to a pass-through. The system becomes a shared trunk plus one recurrent workspace. □

Proposition 5.3 (Channel budget invariant)**.** *If the controller keeps at most $K_{\max}$ live channels after every prune step, then the live channel count remains bounded by $K_{\max}$ at every step.*

Proof. The controller is the only component permitted to alter live-channel count. By contract it outputs no more than $K_{\max}$ channels after each prune step. Induction over steps yields the invariant. □

Proposition 5.4 (Internal search budget bound)**.** *If the controller allows at most B internal refinement rounds per output step and each round preserves the channel budget invariant, then per-token internal search compute is bounded by a function of $K_{\max}$, B, one shared-trunk pass, and one compare/control pass per round.*

Proof. The shared-trunk pass occurs once. Each refinement round is explicitly capped, and each round operates under the preserved channel budget invariant. Therefore the total internal search work is bounded by the declared control-plane budget. □

Proposition 5.5 (No architectural forced collapse under separated slots)**.** *If channel state is separated by explicit slots and cross-channel interaction occurs only through declared compare or merge operators, then the architecture does not itself force branch collapse between those declared interaction points.*

Proof. Each live hypothesis occupies a distinct state slot. Local channel updates act only on that slot, and cross-channel interaction occurs only at explicit operators. Therefore immediate branch collapse is not imposed by the architecture between declared compare or merge steps. □

Proposition 5.6 (Bounded beam-style control skeleton)**.** *A Graph-of-Experts controller with explicit seed, expand, score, prune, merge, and halt operators realizes the same width-bounded successor-selection structure as a beam-style search loop in latent space.*

Proof. Seed initializes the candidate set, expand creates successor states, score ranks them, prune enforces bounded width, merge collapses compatible states when desired, and halt terminates the loop. This is exactly the structural control pattern used by a width-bounded search loop, now expressed over latent channel states rather than explicit symbolic states. □

Proposition 5.7 (Shared-trunk amortization)**.** *Relative to an external GoT harness that re-runs a full model for each branch, a Graph-of-Experts family with one shared trunk amortizes common token processing work across all live channels.*

Proof. The shared token-context processing occurs once in the trunk. Channel-local branching happens after that shared computation, so common prefix processing is not repeated per branch inside the same output step. □

Proposition 5.8 (Token-serialization savings)**.** *A latent-channel model can avoid explicit token serialization of intermediate branches that an external GoT harness would represent as text or extra model calls.*

Proof. Intermediate branch states remain inside the latent frontier F_t rather than being emitted as visible text tokens. Hence the family has a lower token overhead floor than an external text-serialized branch harness. □

Conjecture 5.9 (Search-efficiency advantage over external GoT)**.** *The Graph-of-Experts family may be more token-efficient or more compute-efficient than an external GoT harness because of latent branch storage, shared-trunk amortization, and bounded internal compare/prune loops.*

Conjecture 5.10 (Native-search advantage over plain hybrids)**.** *The family may outperform single-stream hybrids on tasks that require maintaining multiple candidate hypotheses over several reasoning steps.*

6 Boundary

These notes prove or sketch only what can currently be formalized cleanly:

- causality,
- decode equivalence where appropriate,
- reductions,
- structural generalization,
- explicit budget invariants,
- bounded-search expressiveness.

They do not prove benchmark wins, training stability, or emergent reasoning quality. Those remain empirical.